Table of Contents

Asynchronous Operations **1**

API Access **5**

POST Request 6

TextEditingController **10**

Submission Instructions

Please do not upload PDF documents for this course going forward. It is okay of course if you already did.

Asynchronous Operations

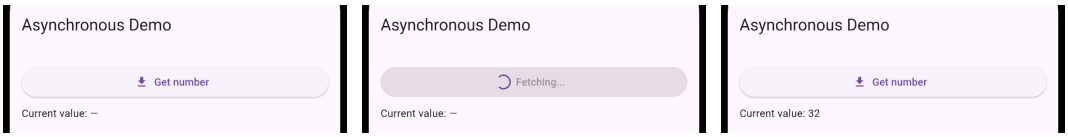
Asynchronous operations allow your program start a task (like reading a file, calling an API, or waiting for a timer) without blocking the rest of the code from running. The task runs in the background so your app stays responsive and can perform other work until the task is completed.

The following structures help with asynchronous operations:

- a **Future<T>** represents a value that arrives later where T is the data type.
- **await** pauses a function until the Future completes so the UI can remain responsive.
- To allow functions to contain await instructions, they must be marked with **async**.
- **try / catch** to are helpful for catching async errors gracefully without a program crash.

Example 1: Asynchronous Operations

This example demonstrates how to enable a basic asynchronous event. To keep the application simple, a simulated delay of 2 seconds is implemented and a number (current seconds) is returned when it when the wait is finished. A circular progress indicator is displayed while the app waits for the task to complete.



main.dart

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: const AsyncDemoScreen(),
    );
  }
}

class AsyncDemoScreen extends StatefulWidget {
  const AsyncDemoScreen({super.key});
  @override
  State<AsyncDemoScreen> createState() => _AsyncDemoScreenState();
}

class _AsyncDemoScreenState extends State<AsyncDemoScreen> {
  int?     _currentValue;
  bool     _loading = false;
  String?  _error;

  /// Simulate an async operation (e.g., HTTP call or database read).
  Future<int> _getNumber() async {
    await Future.delayed(const Duration(seconds: 2));
    return DateTime.now().second;
  }

  Future<void> _updateNumber() async {
    setState(() {
      _loading = true;
      _error   = null;
    });
    try {
      final result = await _getNumber(); // <-- await pauses only this function
      if (!mounted) return; // prevent issue if widget is disposed during await
      setState(() {
        _currentValue = result;
      });
    } catch (e) {
      setState(() {
        _error = e.toString();
      });
    } finally {
```

Exercise 1 (1 mark)

For _getNumber(), what error appears for the await operator if async operator is deleted?

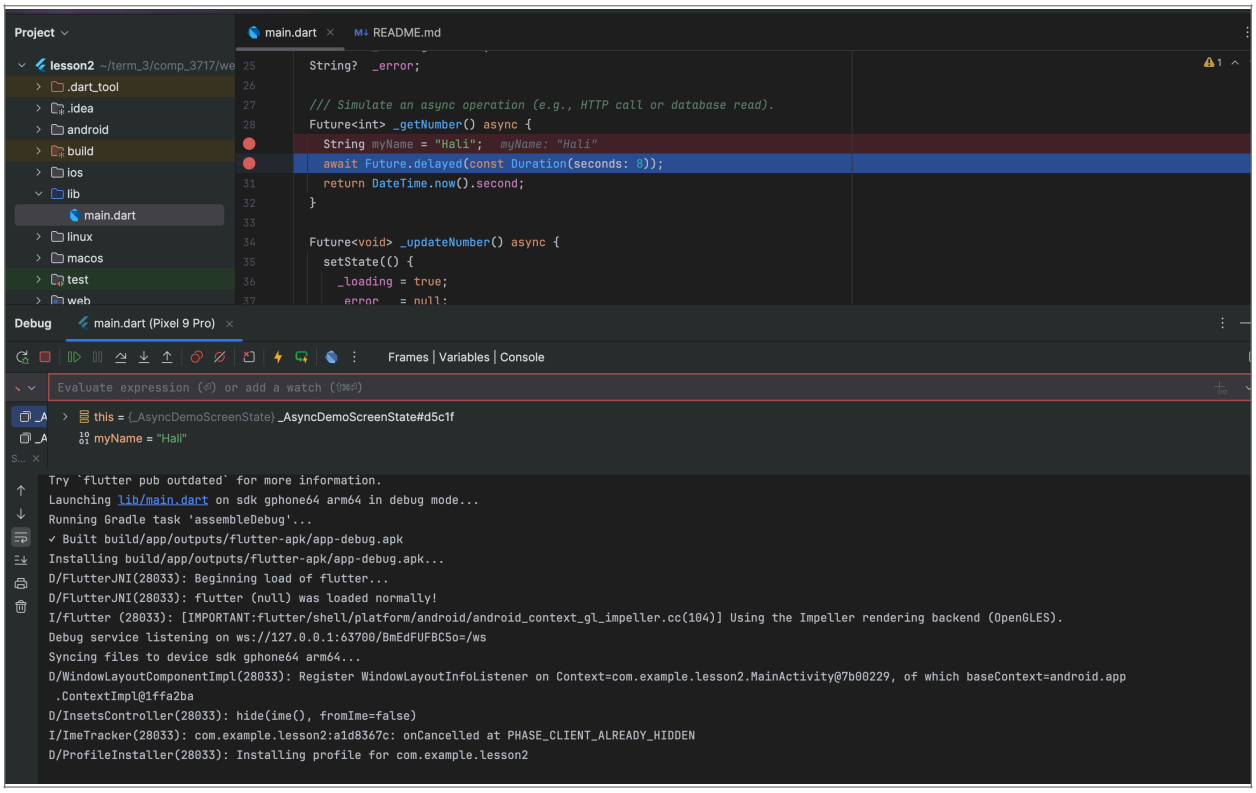
await' can only be used in 'async' or 'async*' methods.

Exercise 2 (2 marks)

Inside _getNumber(), declare a variable with your name. Change the wait time to 8 seconds. Then set two breakpoints inside as shown below. The screenshot should look like this but with your name in it.

```
28  Future<int> _getNumber() async {
29    String fname = "Pat";   fname: "Pat"
      await Future.delayed(const Duration(seconds: 8));
      return DateTime.now().second;
32  }
```

Take a screenshot while halted at the second breakpoint so your name and both breakpoints are visible. Show the screenshot here.



Exercise 3 (2 marks)

Modify _getNumber() so it contains a divide by zero error and is set to wait 8 seconds.

```
Future<int> _getNumber() async {
  await Future.delayed(const Duration(seconds: 8));
  double x = 100/0;
  return x.toInt();
}
```

Next, declare variables that store your first and last name in the catch and finally blocks respectively. Then set break points in both the catch and finally blocks. Take screenshots of the code window while halted in the catch and finally blocks. The two screenshots should look like the following:

```
46        setState(() {
47          String fname = "Pat";   fname: "Pat"
          _error = e.toString();
49        });
50      } finally {
51        setState(() {
52          String lname = "McGee";
          _loading = false;
54        });
55      }
```

```
46        setState(() {
47          String fname = "Pat";
          _error = e.toString();
49        });
50      } finally {
51        setState(() {
52          String lname = "McGee";   lname: "McGee"
          _loading = false;
54        });
55      }
```

Show the screenshot while halted in the catch block with your name in it.

$

```
main.dart ×   M↓ README.md
42        setState(() {
43          _currentValue = result;
44        });
45      } catch (e) {   e: Unsupported operation: Infinity or NaN toInt
        String firstName = "Hali";
47        setState(() {
48          _error = e.toString();
49        });
50      } finally {
        String lasttName = "Imanpanah";
52        setState(() {
53          _loading = false;
54        });
55      }
56    }
57
58    @override
59    Widget build(BuildContext context) {
60      return Scaffold(
61        appBar: AppBar(title: const Text('Asynchronous Demo')),
62        body: Padding(
```

Show the screenshot while halted in the finally block with your name in it.

```
main.dart ×   README.md
42        setState(() {
43          _currentValue = result;
44        });
45      } catch (e) {
46        setState(() {
47          String firstName = "Hali";
48          _error = e.toString();
49        });
50      } finally {
51        setState(() {
52          String lasttName = "Imanpanah";
53          _loading = false;
54        });
55      }
56    }
57
58    @override
59    Widget build(BuildContext context) {
60      return Scaffold(
61        appBar: AppBar(title: const Text('Asynchronous Demo')),
62        body: Padding(
```

What conclusions can you make about the finally block?

Finally block always executes after the try-catch, regardless of whether an error occurred. This makes it useful for cleanup code that must run in both success and error scenarios.

Exercise 4 (1 mark)

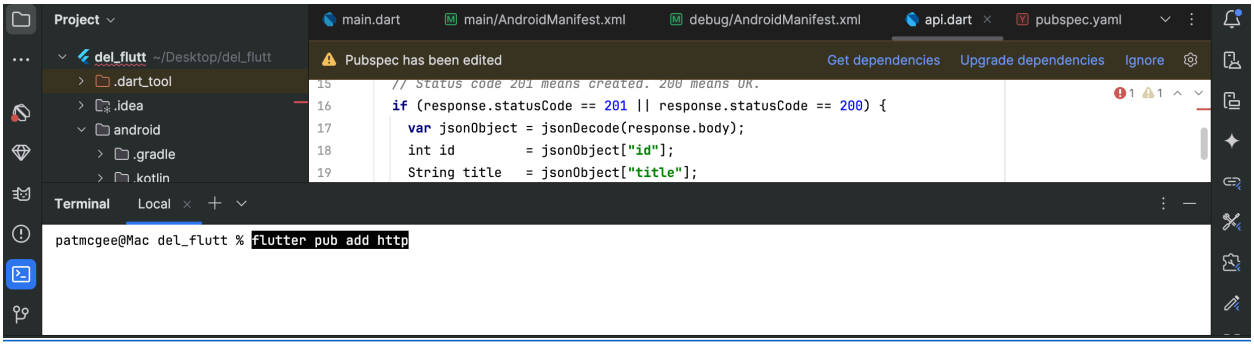
Starting with Exercise 3, delete the await keyword from the _getNumber() function. After making this change, how long do you have to reach the second break point?

Immediately, 0 second

API Access

Accessing remote 3rd party API's requires asynchronous handling so the application does not freeze while waiting for a response. To access API's with http requests from a Flutter app, the http dependency can be added by running the following command in the terminal:

```
flutter pub add http
```

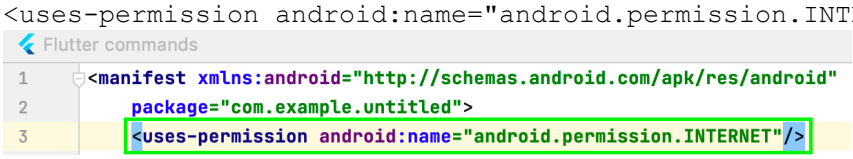


Tip: If flutter is not recognized on your Mac then run these commands in the terminal. Be sure to use the location of your flutter folder:

export FLUTTER_HOME="/Users/patmcgee/development/flutter"
export PATH="$FLUTTER_HOME/bin:$PATH"

If you develop for Android, add the following permission inside manifest tag in the AndroidManifest.xml file located at **android/app/src/main**. The permission may already be present by default.

```
<uses-permission android:name="android.permission.INTERNET"/>
```

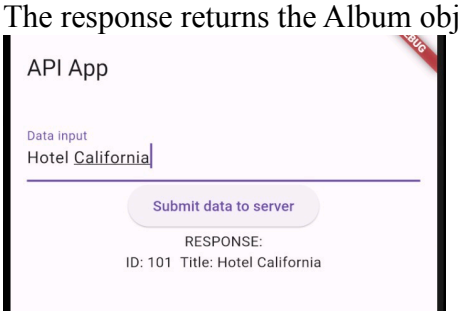


POST Request

Example 2: POST Requests

This example shows how to enable a POST request which posts Album title information to https://jsonplaceholder.typicode.com/albums

The response returns the Album object which includes id and title attributes.



api.dart uses async / await structures to retrieve the data asynchronously. A **Future<String>** type indicates the data type that will be returned in the future so the rest of the program can function properly without freezing until the data is delivered from another thread.

lib\api.dart

```
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<String> createAlbum(String inputString) async {
  final response = await http.post(
    Uri.parse('https://jsonplaceholder.typicode.com/albums'),

    headers: <String, String>{
      'User-Agent': 'MyFlutterApp/1.0 (Flutter; Android)',
      'Content-Type': 'application/json; charset=UTF-8',
    },
    body: jsonEncode(<String, String>{
      'title': inputString,
    }),
  );
  // Status code 201 means created. 200 means OK.
  if (response.statusCode == 201 || response.statusCode == 200) {
    var jsonObject = jsonDecode(response.body);
    int id         = jsonObject["id"];
    String title   = jsonObject["title"];
    String idStr   = id.toString();
    return "ID: $idStr  Title: $title";
  }
  else { // Response not received.
    final errorMessage
    = 'Failed to create Album. Error: $response.statusCode - $response.body';
    return errorMessage;
  }
}
```

lib\main.dart

```dart
import 'package:flutter/material.dart';
import 'api.dart';

void main() {
  runApp(MaterialApp(home: MyApp()));
}

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  MyStateFullApp createState() => MyStateFullApp();
}

class MyStateFullApp extends State<MyApp> {
  // Set text here.
  String _inputTitle = '';
  String _title      = 'empty';

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("API App"),
      ),
      body: SingleChildScrollView( // Avoids screen overflow
        child: Padding(
          padding: const EdgeInsets.all(16.0), // Add padding around the Column
          child: Column(
            children: <Widget>[
              TextField(
                  onChanged: (String text){
                    setState(() {
                      _inputTitle = text;
                    });
                  },
                  decoration: const InputDecoration(
                      hintText: "Enter some data", labelText: "Data input")),

              ElevatedButton(
                  onPressed: () async {
                    String newAlbumData = await createAlbum(_inputTitle);
                    setState(() {
                      _title = newAlbumData;
                    });
                  },
                  child:  const Text('Submit data to server')),
              const Text("RESPONSE:"),
```

Exercise 5 (1 mark)

How is the Button's onPressed() callback enabled to make a call for an asynchronous resource?

By making the onPressed callback async (using the async keyword), it can use await to call and wait for asynchronous resources like the createAlbum() function.

Exercise 6 (2 marks)

Starting with Example 2, set a breakpoint where the jsonObject is returned. Then, submit your first name to the server. While halted at the breakpoint, expand the jsonObject in the debugger so it shows your name in the response. Your screenshot should appear similar to the one below but it will display your name.

Show your screenshot here:

```
16    // Status code 201 means created. 200 means OK.
17    if (response.statusCode == 201 || response.statusCode == 200) {
18      var jsonObject = jsonDecode(response.body);   jsonObject: size = 2
      int id          = jsonObject["id"];
20      String title    = jsonObject["title"];
21      String idStr    = id.toString();
22      return "ID: $idStr  Title: $title";
23    }
```

main.dart (Pixel 9 Pro)

Frames | Variables | Console

Evaluate expression (⏎) or add a watch (⇧⌘⏎)

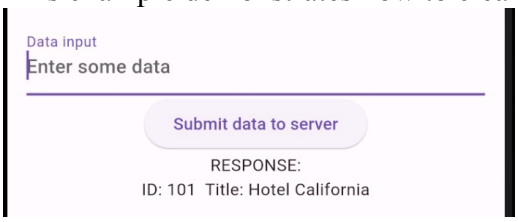
```
inputString = "Hali"
response = {Response}
jsonObject = {_Map} size = 2
  0 = {map entry} "title" -> "Hali"
  1 = {map entry} "id" -> 101
```

TextEditingController

TextField objects may include a controller attribute that can be used to reset its text with a **clear()** function. The **TextEditingController** type defines the object for this attribute.

Example 3: Clearing the TextField with a TextEditingController

This example demonstrates how to clear a TextField after data is submitted to the server.



To enable automatic clearing of the TextField in Example 2, use the following steps. In the State class, add this class level declaration for the controller:

```
final TextEditingController _controller = TextEditingController();
```

Then in the TextField constructor, add the controller attribute:

```
controller: _controller,
```

After, in the onPressed() handler for the button, add this instruction at the end of onPressed() to clear the TextField:

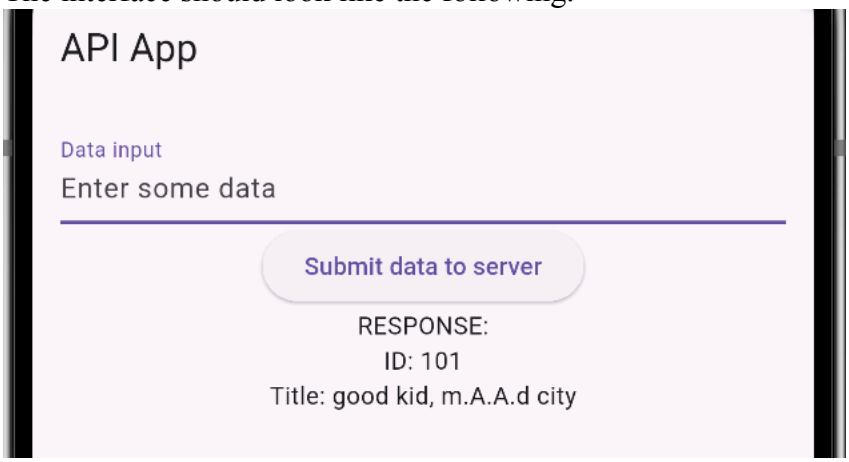
```
_controller.clear();
```

Exercise 7 (3 marks)

Define a class named Album to store the id and title attributes. Use a constructor that requires named parameters (refer to labWk1). Then, when the data is retrieved, store the id and title in an Album object. The return type of the function header for createAlbum() must be changed to:

```
Future<Album> createAlbum(String inputString) async
```

The interface should look like the following.



After jsonDecode is called, initialize an album object. Set a breakpoint after the album object is initialized. Halt after the album is initialized and show the attribute values while halted in the debugger.

If an error occurs you may return:

```
  return Album(id: -1, title: errorMessage);
```

Or you could throw the exception:

```
  throw Exception(errorMessage);
```

Show a screenshot of the album object while halted in the debugger:

```
31
    return album;
33  }
34  else { // Response not received.
35    final errorMessage
36    = 'Failed to create Album. Error: $response.statusCode - $respons
37    return Album(id: -1, title: errorMessage);
38  }
39 }
```

main.dart (Pixel 9 Pro)

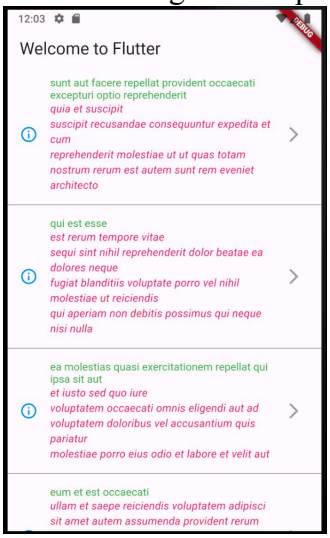
Frames | Variables | Console

Evaluate expression (⏎) or add a watch (⇧⌘⏎)

```
inputString = "Hali"
response = {Response}
jsonObject = {_Map} size = 2
id = 101
title = "Hali"
album = {Album}
    id = 101
    title = "Hali"
```

Example 4: GET Requests

The following code implements GET requests in Flutter.



Since the data is delivered asynchronously in a list of objects that are of an unknown type, a **Future<List<dynamic>>** type is used.

lib\api.dart

```
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<List<dynamic>> getNews() async {
  String url       = "https://jsonplaceholder.typicode.com/posts";
  final response   = await http.get(Uri.parse(url),
    headers: {
      'User-Agent': 'MyFlutterApp/1.0 (Flutter; Android)',
      'Accept': 'application/json',
    },
  );

  if (response.statusCode == 201 || response.statusCode == 200) {
    var responseData = jsonDecode(response.body); // Get response data
    return responseData; // Return the response data
  } else {
    final errorMessage =
        'Failed to create Album. Error: ${response.statusCode} - ${response.body}';
    throw Exception(errorMessage);
  }
}
```

Inside the State class, a Future async method is used to invoke the getNews() function which calls the API asynchronously.

```
Future<void> fetchData() async {
    var data = await getNews();
    setState(()  {
        _newsData = data;
    }); // Update the UI after fetching data
  }
```

The fetchData( ) method is called from the initState() method override which fetches the data as soon as the widget has been built. If fetchData() were called from the build() function, the update to the state variable would cause continuous unnecessary rebuilds.

```
  @override
  void initState() {
      super.initState(); // More proper to ensure parent correctly initializes state.
      fetchData();
  }
```

lib\main.dart

```
import 'package:flutter/material.dart';
import 'dart:async';
import 'api.dart';

// 1. The program starts here.
void main() {
  runApp( MaterialApp(home: MyApp()));
}

// 2. This step feels inefficient but we need this extra step to
//    launch a widget that can manage state objects.
class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  MyStateFullApp createState() =>
       MyStateFullApp(); // Launch stateful component.
}

// 3. This stateful component enables class level variables.
class MyStateFullApp extends State<MyApp> {
  var _newsData;

  @override
  void initState() {
    // Make sure parent class is correctly used to
    // correctly set up the widget's state.
    super.initState();

    // Call the async method to fetch data as
    // soon as the widget is in the tree.

    // If this was called in the build() method, it would
    // force a continuous rebuild.
    fetchData();
  }

  Future<void> fetchData() async {
    var data = await getNews();
    setState(()  {
      _newsData = data;
    }); // Update the UI after fetching data
  }

  @override
  Widget build(BuildContext context) {

    return MaterialApp(
```

Exercise 8 (3 marks)

Starting with Example 4, place the following print statements in the app.

- Inside an empty default constructor for MyStateFullApp state object add the following code:

```
print("*** Inside MyStateFullApp constructor. Mounted status: $mounted");
```

- At the end of initState() add the following code:

```
print("*** Inside initState(). Mounted status: $mounted");
```

- At the very start of build() add the following code:

```
print("*** Inside build(). Mounted status: $mounted");
```

Run the code and examine the trace statements. In Flutter, what does mounted indicate?

mounted indicates whether the State object is currently attached to the widget tree. When true, the widget is active and can call setState()

Why does build() appear twice in the trace statements?

first to show the loading indicator when _newsData is null, and second after fetchData() completes and calls setState() to display the data.

Exercise 9 (2 marks)

Starting with the solution from Exercise 8, move the fetchData() instruction so it is just inside the build() function. Run the program and show of the trace output here.

```
I/flutter ( 6233): *** Inside MyStateFullApp constructor. Mounted status: false
I/flutter ( 6233): *** Inside initState(). Mounted status: true
I/flutter ( 6233): *** Inside build(). Mounted status: true
D/InsetsController( 6233): hide(ime(), fromIme=false)
I/ImeTracker( 6233): com.example.lesson2:ce5c7dc: onCancelled at
PHASE_CLIENT_ALREADY_HIDDEN
I/flutter ( 6233): *** Inside build(). Mounted status: true
I/flutter ( 6233): *** Inside build(). Mounted status: true
I/flutter ( 6233): *** Inside build(). Mounted status: true
I/flutter ( 6233): *** Inside build(). Mounted status: true
I/flutter ( 6233): *** Inside build(). Mounted status: true
I/flutter ( 6233): *** Inside build(). Mounted status: true
I/flutter ( 6233): *** Inside build(). Mounted status: true
I/flutter ( 6233): *** Inside build(). Mounted status: true
I/flutter ( 6233): *** Inside build(). Mounted status: true
I/flutter ( 6233): *** Inside build(). Mounted status: true
```

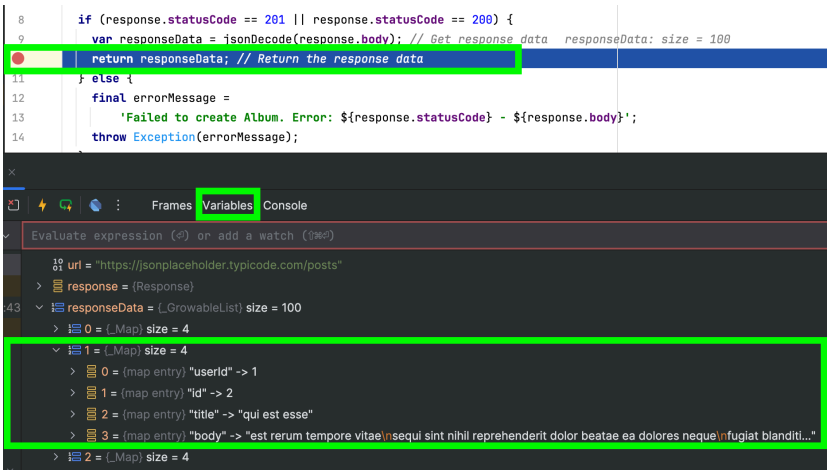
Explain what you are observing in the trace output.

Calling fetchData() in build() triggers setState(), which causes build() to run again, creating an infinite loop of rebuilds.

Exercise 10 (2 marks)

Starting with Example 4, place a breakpoint inside getNews(). Then run the program. While halted at the return statement, inspect the responseData variable. The content on the **Variables** tab should be similar to my screenshot below. However, expand the first item in the list so all four of it's attributes are visible.

Take a screenshot of the first element's four attributes when they are visible in the debugger. This screenshot shows the second element's four attributes.



Show the screenshot of the first element's four attributes in the debugger here:

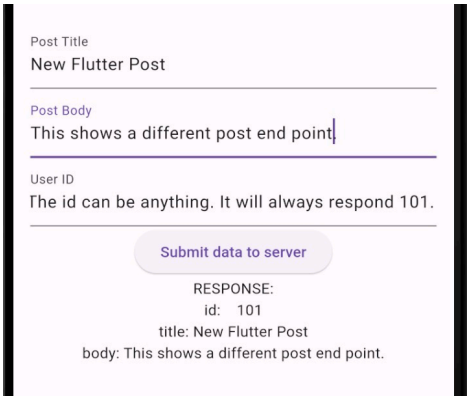
An exercise in the lab will require more work with HTTP requests and API.

Exercise 11 (8 marks)

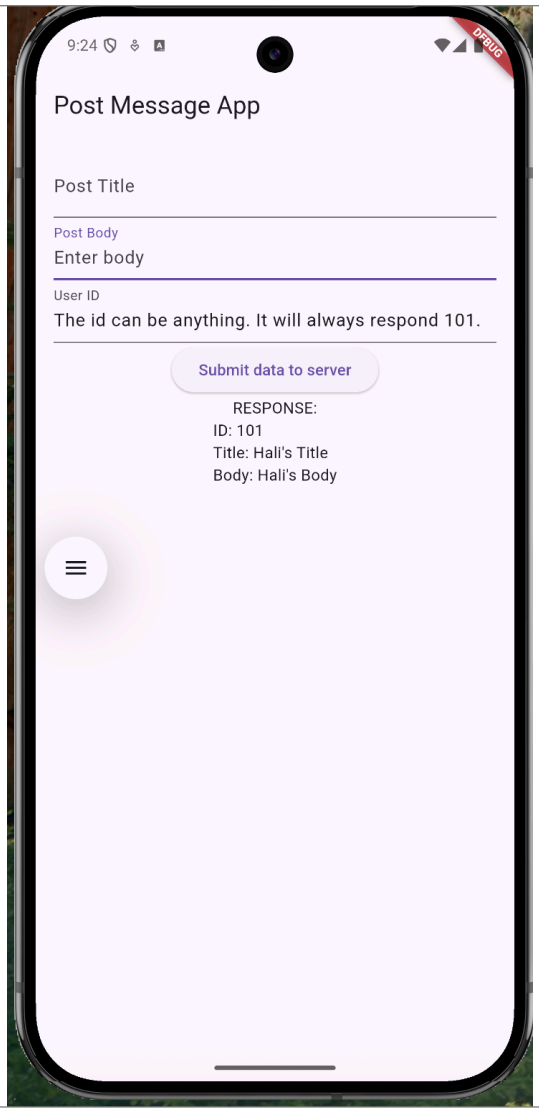
Starting with Example 2 and the following hint code, modify the application to allow users to submit a message post title and body to https://jsonplaceholder.typicode.com/posts

```
final response = await http.post(
  Uri.parse('https://jsonplaceholder.typicode.com/posts'),
  headers: <String, String>{
    'Content-Type': 'application/json; charset=UTF-8',
  },
  body: jsonEncode({
    'title': title,
    'body':body,
    'id':"This can be anything. The response is always id=101.",
  }),
);
```

Enable an interface like the following.



Show your screenshot of your finished working interface here with your name in the response:



Exercise 12 (12 marks)

Starting with Example 4, change the URL to retrieve this single message post with a GET request.
https://jsonplaceholder.typicode.com/comments?postId=3&id=12

Check with the debugger to examine the data that is returned right after the response body is decoded. The data should look similar to the following.

```
[
  {
    "postId": 3,
    "id": 12,
    "name": "modi ut eos dolores illum nam dolor",
    "email": "Oswald.Vandervort@leanne.org",
    "body": "expedita maiores dignissimos facilis\nipsum est rem est fugit velit sequi\neum
odio dolores dolor totam\noccaecati ratione eius rem velit"
  }
]
```

Create a Message class which contains the attributes shown in the screenshot above. Convert the getNews() function so it returns a single Message object instead of a list.

Hints:

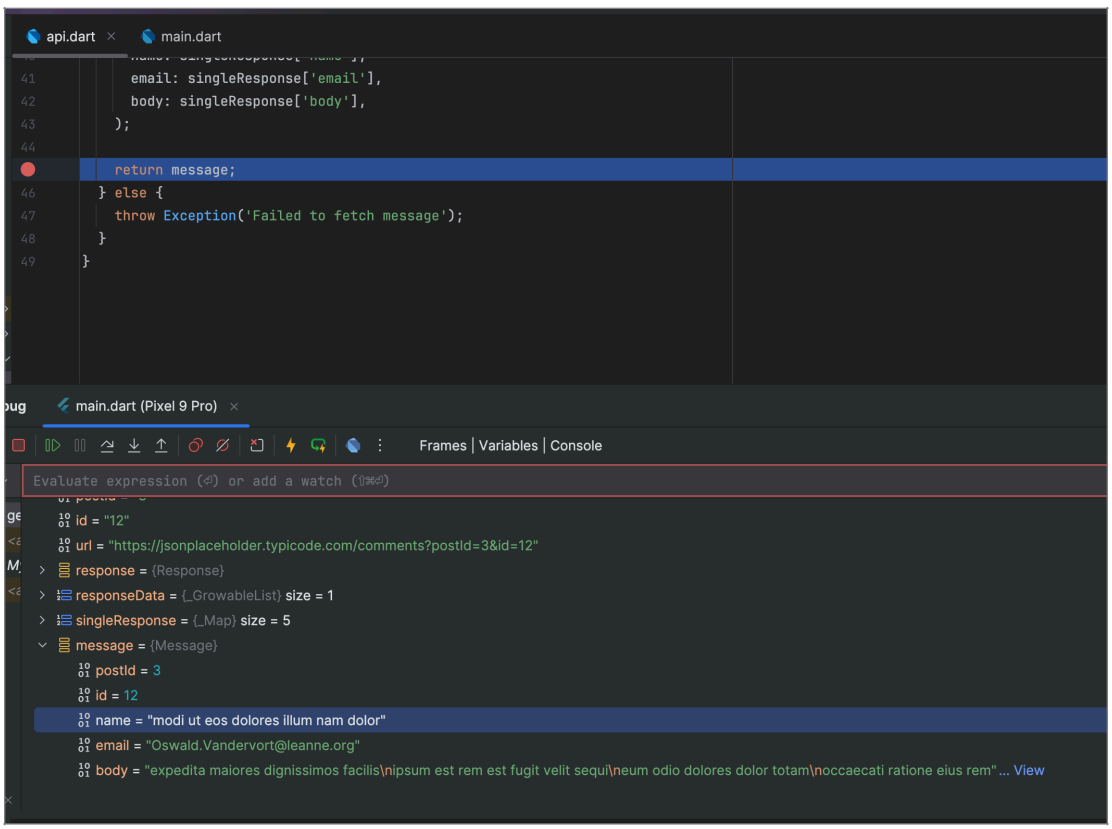
The getNews() function header should by the following:

```
Future<Message> getNews(String id, String postID) async
```

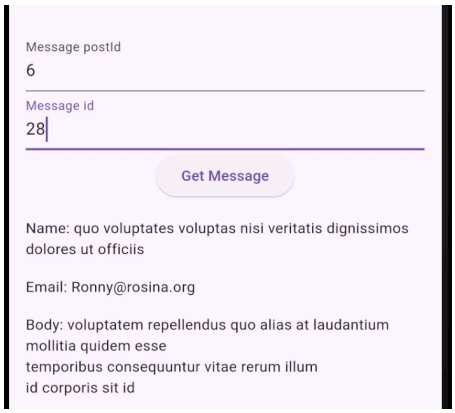
The treat the item as a single object it must be removed from the array. This could be done with the following lines of code:

```
var responseData = jsonDecode(response.body); // Get response data
var singleResponse = responseData[0];
```

a) Show a screenshot of the debugger while halted inside the api.dart file after the response has been converted to a Message object. Show the contents of the Message object as displayed in the debugger. (6 marks)



b) Build an interface that looks like the following to allow users to enter a **postId** and **id** to retrieve and view a single message. (8 marks)

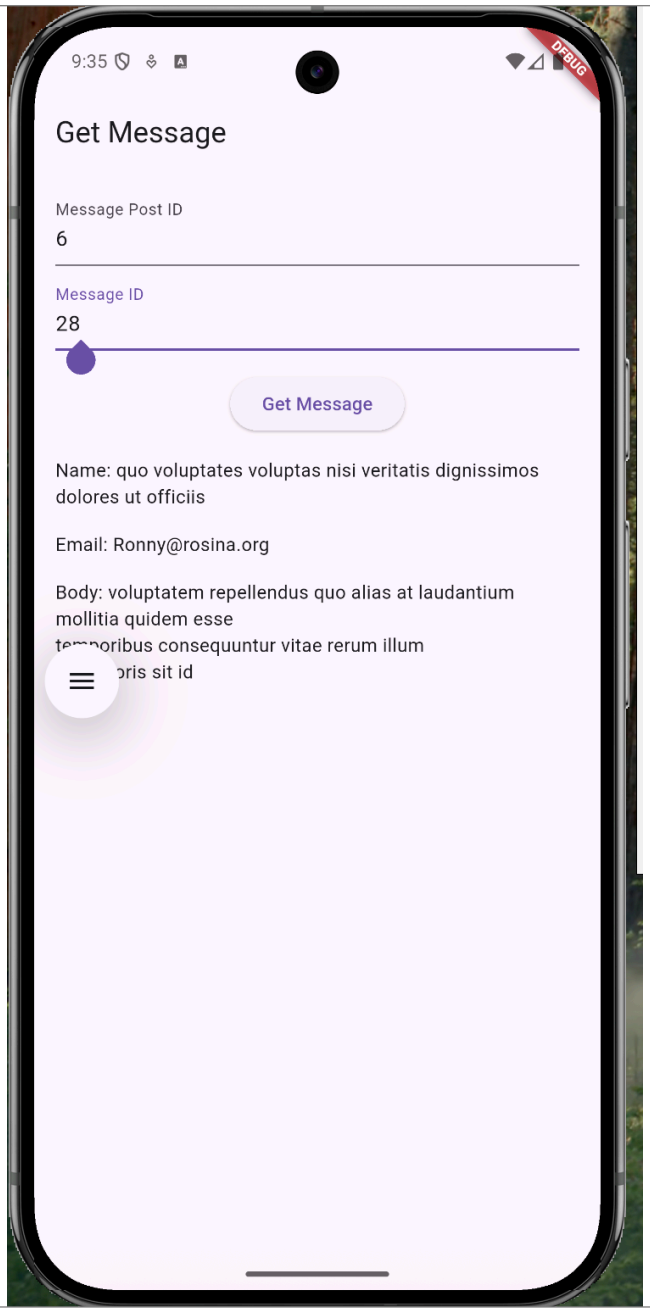


For a list of available postId and id combinations see https://jsonplaceholder.typicode.com/comments

For hints on how to build the interface see Example 2 in main.dart. Left alignment of the name, body and email data was achieved with the following widgets:

```
SizedBox(height: 16.0),
Align(
  alignment: Alignment.centerLeft,
  child: Text("Name: " + _message.name),
),
```

Show a screenshot of your interface after implementing using it to retrieve a message with a different **postId** and **id**. Error handling is not required since this task will be time consuming enough. Show your screenshot here:


9:35
DEBUG
Get Message
Message Post ID
6
Message ID
28
Get Message
Name: quo voluptates voluptas nisi veritatis dignissimos dolores ut officiis
Email: Ronny@rosina.org
Body: voluptatem repellendus quo alias at laudantium mollitia quidem esse
temporibus consequuntur vitae rerum illum
oris sit id